Análisis de Patrones Creacionales en Spring Framework

Diego Alejandro Triana Paredes

02230131049

Universidad de Santander – UDES

Ingeniería de Sistemas Patrones de Diseño de Software Unidad 2: Patrones Creacionales Post-Contenido 2

22 de marzo de 2026

Introducción

Spring Framework es uno de los frameworks de desarrollo de software más utilizados en el ecosistema Java empresarial. Su arquitectura interna está construida sobre una base sólida de patrones de diseño que permiten gestionar de forma eficiente la creación, configuración y ciclo de vida de los objetos dentro de una aplicación. Esto lo convierte en un caso de estudio especialmente relevante para comprender cómo los patrones creacionales del catálogo GoF se aplican en contextos reales y a escala industrial.

El análisis de los patrones creacionales en Spring permite entender que estos no son simplemente conceptos teóricos, sino soluciones concretas a problemas reales de instanciación y configuración de objetos. En este documento se analizan tres patrones creacionales identificados en el código fuente de Spring Framework: Singleton, Factory Method y Builder, describiendo para cada uno la clase concreta donde aparece, el problema que resuelve y el principio SOLID al que está asociado.

Análisis de Patrón 1: Singleton

1. ¿Cuál es el patrón y a qué categoría GoF pertenece?

El patrón Singleton pertenece a la categoría de patrones creacionales del catálogo GoF. Su propósito es garantizar que una clase tenga una única instancia durante todo el ciclo de vida de la aplicación, proporcionando un punto de acceso global a dicha instancia. Este patrón es especialmente útil cuando se necesita coordinar acciones a través de un único objeto compartido.

2. ¿Dónde aparece en Spring Framework?

En Spring Framework, el patrón Singleton se implementa en el módulo spring-core, específicamente en la clase:

org.springframework.beans.factory.support.DefaultSingletonBeanRegistry

Esta clase implementa la interfaz SingletonBeanRegistry y es la responsable de registrar y gestionar todas las instancias singleton de los beans dentro del contenedor IoC de Spring.

3. ¿Qué problema resuelve en este contexto?

En una aplicación empresarial, múltiples componentes necesitan acceder a los mismos servicios, repositorios o configuraciones. Sin un mecanismo centralizado, cada solicitud de un bean crearía una nueva instancia, generando un consumo innecesario de memoria y posibles inconsistencias de estado. Spring resuelve esto garantizando que, por defecto, cada bean registrado en el contenedor tenga una única instancia compartida por toda la aplicación.

4. ¿Qué evidencia de código lo confirma?

```java
// Clase: DefaultSingletonBeanRegistry
// Paquete: org.springframework.beans.factory.support
// Módulo: spring-beans

// Mapa que almacena las instancias singleton ya creadas
private final Map<String, Object> singletonObjects =
        new ConcurrentHashMap<>(256);

// Método que retorna la instancia singleton registrada
@Override
@Nullable
public Object getSingleton(String beanName) {
    // Busca primero en el caché de singletons ya inicializados
    return getSingleton(beanName, true);
}
```

Este mapa singletonObjects actúa como el registro central de instancias. Una vez que un bean es creado, se almacena aquí y todas las solicitudes posteriores retornan la misma instancia sin crear una nueva.

5. ¿Qué principio SOLID refuerza?

Este patrón refuerza el principio de Responsabilidad Única (SRP) al centralizar en una sola clase toda la lógica de gestión del ciclo de vida de las instancias singleton. También apoya el principio de Inversión de Dependencias (DIP), ya que los componentes de la aplicación dependen de abstracciones registradas en el contenedor y no crean sus propias instancias.

Análisis de Patrón 2: Factory Method

1. ¿Cuál es el patrón y a qué categoría GoF pertenece?

El patrón Factory Method pertenece a la categoría de patrones creacionales del catálogo GoF. Define una interfaz para crear objetos, pero delega a las subclases la decisión de qué clase concreta instanciar. Permite que el código cliente trabaje con abstracciones sin conocer las clases concretas que se están creando.

2. ¿Dónde aparece en Spring Framework?

En Spring Framework, el patrón Factory Method se implementa en el módulo spring-context, en la interfaz:

`org.springframework.beans.factory.BeanFactory`

Y su implementación más relevante es:

`org.springframework.context.annotation.ConfigurationClassBeanDefinitionReader`

Adicionalmente, los métodos anotados con `@Bean` dentro de clases `@Configuration` son en esencia factory methods que Spring invoca para crear las instancias de los beans.

3. ¿Qué problema resuelve en este contexto?

Spring necesita crear beans de distintos tipos según la configuración de la aplicación, sin que el código cliente conozca las clases concretas. Si el framework instanciara directamente cada clase con `new`, quedaría acoplado a implementaciones específicas y sería muy difícil de extender o reemplazar componentes. El Factory Method permite desacoplar la creación de objetos de su uso.

4. ¿Qué evidencia de código lo confirma?

```java
// Interfaz: BeanFactory
// Paquete: org.springframework.beans.factory
// Módulo: spring-beans

public interface BeanFactory {

    // Factory Method: crea o retorna el bean del tipo solicitado
    // El cliente solo conoce el nombre o tipo, no la clase concreta
    Object getBean(String name) throws BeansException;

    // Variante tipada del factory method
    <T> T getBean(String name, Class<T> requiredType)
            throws BeansException;
}
```

Cuando el desarrollador define un método con `@Bean` en una clase `@Configuration`, Spring lo trata internamente como un factory method y lo invoca mediante proxies CGLIB para garantizar que se respete el scope del bean.

5. ¿Qué principio SOLID refuerza?

Este patrón refuerza el principio Abierto/Cerrado (OCP), ya que es posible agregar nuevos tipos de beans sin modificar el código del contenedor. También apoya el principio de Inversión de Dependencias (DIP), porque el código cliente depende de la interfaz `BeanFactory` y no de implementaciones concretas.

Análisis de Patrón 3: Builder

1. ¿Cuál es el patrón y a qué categoría GoF pertenece?

El patrón Builder pertenece a la categoría de patrones creacionales del catálogo GoF. Su propósito es separar la construcción de un objeto complejo de su representación, permitiendo crear diferentes configuraciones del mismo objeto usando el mismo proceso de construcción. Es especialmente útil cuando un objeto requiere múltiples pasos de configuración antes de estar listo para usarse.

2. ¿Dónde aparece en Spring Framework?

En Spring Security, el patrón Builder se implementa de forma prominente en la clase:

`org.springframework.security.config.annotation.web.builders.HttpSecurity`

Y también en:

`org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder`

Ambas clases forman parte del módulo spring-security-config.

3. ¿Qué problema resuelve en este contexto?

La configuración de seguridad en una aplicación web involucra muchos parámetros: reglas de autorización, mecanismos de autenticación, manejo de sesiones, protección CSRF, entre otros. Crear un objeto `SecurityFilterChain` directamente con un constructor requeriría pasar todos estos parámetros a la vez, haciendo el código ilegible y propenso a errores. El Builder permite configurar cada aspecto de forma independiente y encadenada, construyendo el objeto final solo cuando está completamente configurado.

4. ¿Qué evidencia de código lo confirma?

```java
// Clase: HttpSecurity
// Paquete: org.springframework.security.config.annotation.web.builders
// Módulo: spring-security-config

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

    // Patrón Builder: cada llamada configura un aspecto diferente
    // y retorna la misma instancia para encadenar llamadas (fluent API)
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login").permitAll()
        )
        .logout(logout -> logout.permitAll());

    // El objeto final se construye al llamar build()
    return http.build();
}
```

Cada método encadenado configura una parte del objeto. El método `build()` al final es el que construye y retorna el objeto completamente configurado, siguiendo la estructura clásica del patrón Builder.

5. ¿Qué principio SOLID refuerza?

Este patrón refuerza el principio de Responsabilidad Única (SRP), ya que cada método del builder se encarga de configurar un único aspecto del objeto. También apoya el principio Abierto/Cerrado (OCP), porque es posible agregar nuevas configuraciones sin modificar el proceso de construcción existente.

Conclusiones

El análisis del código fuente de Spring Framework permitió identificar que los patrones creacionales no son recursos académicos aislados, sino soluciones técnicas concretas que resuelven problemas reales en el desarrollo de software a gran escala. Los tres patrones analizados cumplen roles complementarios dentro del framework: el Singleton garantiza la unicidad y el control del ciclo de vida de los beans, el Factory Method desacopla la creación de objetos de su uso permitiendo extender el sistema sin modificar el código existente, y el Builder simplifica la construcción de objetos complejos con múltiples parámetros de configuración.

Se observa también que cada patrón está directamente relacionado con uno o más principios SOLID, lo que confirma que los patrones de diseño y los principios de diseño orientado a objetos son complementarios y se refuerzan mutuamente. Spring Framework demuestra que aplicar estos conceptos de forma consistente resulta en un sistema altamente extensible, mantenible y fácil de configurar, cualidades que explican en gran medida la adopción masiva del framework en la industria del software empresarial.

Referencias

Pivotal Software. (2024). *Spring Framework Reference Documentation — Core: The IoC Container*. https://docs.spring.io/spring-framework/reference/core/beans.html

Shvets, A. (2024). *Patrones de diseño creacionales*. Refactoring Guru. https://refactoring.guru/es/design-patterns/creational-patterns

Spring Projects. (2024). *spring-framework* [Repositorio de código fuente]. GitHub. https://github.com/spring-projects/spring-framework